Transfer learning, Object Localization and Object Detection

Yolo → v3

Object Detection and Basic Object Detection Algorithms

Some of the Building Bocks for Object Detection

Courtesy: Andrew NG

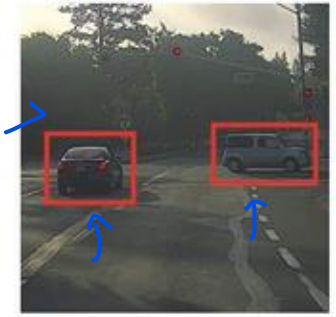
What is Object localization and Detection?

- Object Detection
- Object Localization

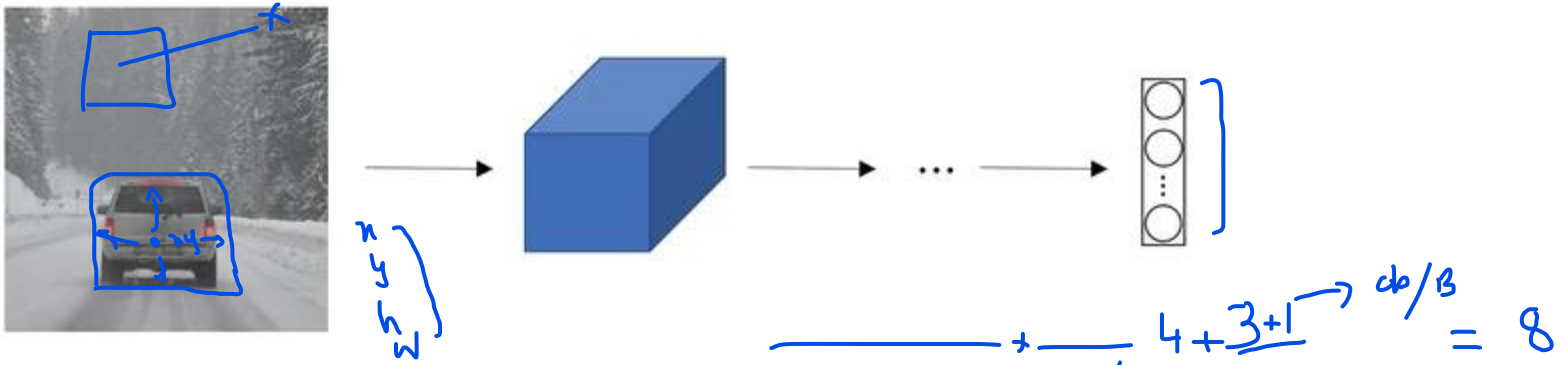
Image classification

Classification with localization

Detection

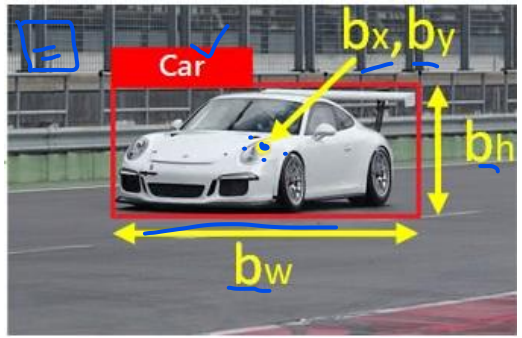


Classification with Localization



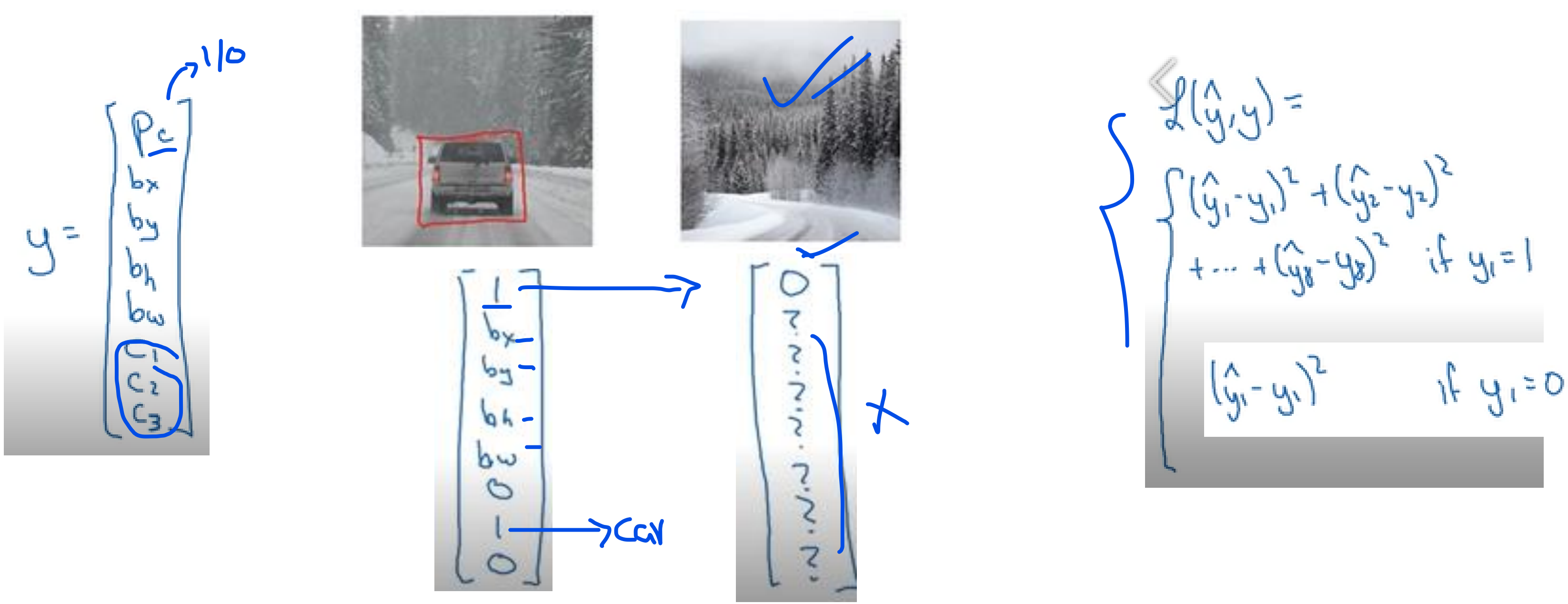
- Object categories for self-driving car
  1. Car 2. Bike, 3. Truck, 4.Background
- Softmax with 4 possible outputs and

4 output units for bounding box (bx,by, bh, bw)



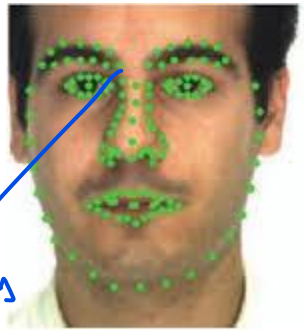
How to define the target label Y

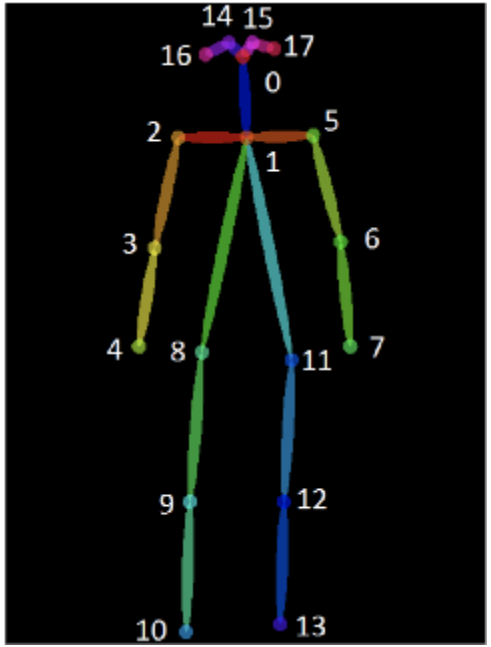
- 4 Object categories: 1. Car 2. Bike, 3. Truck, 4.Background
- Need to output bx, by, bh, bw, and class labels (1-4)



Landmark Detection

b_x, b_y, b_h, b_w





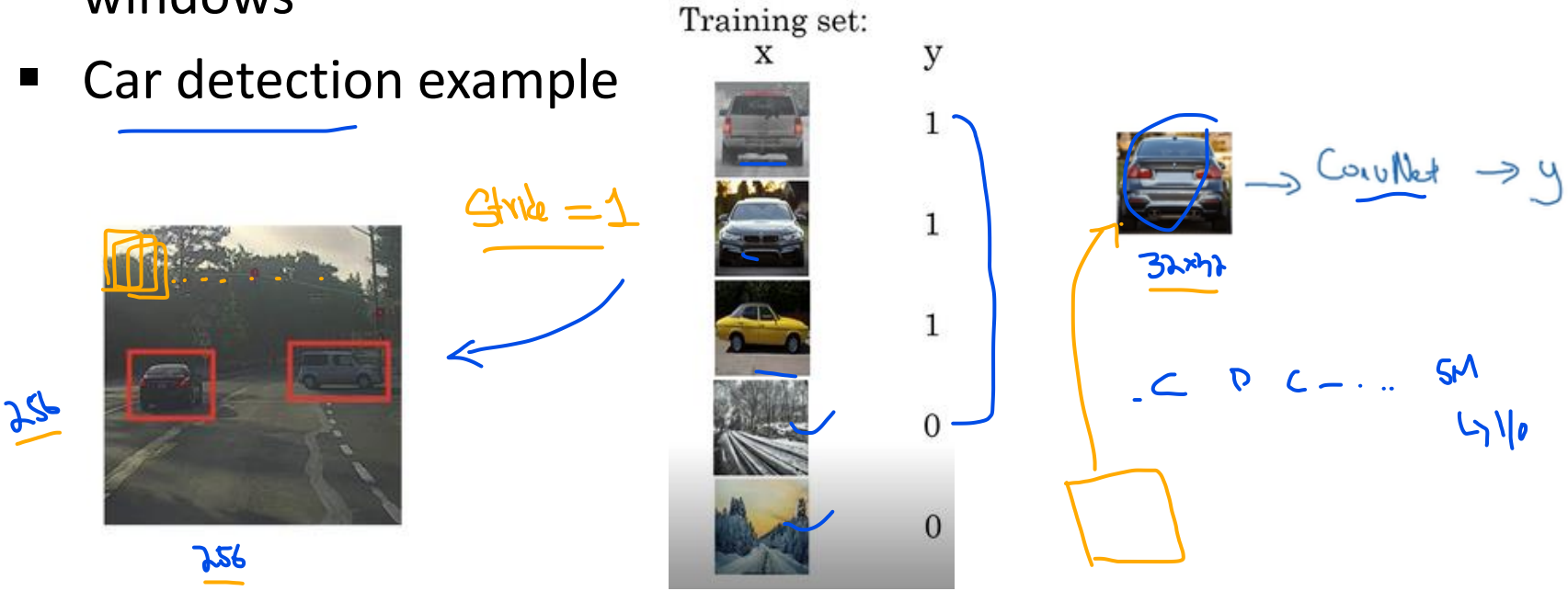
Face

$1 + (4 \times 2)$

$$\begin{bmatrix} l_{1x}, l_{1y} \\ l_{2x}, l_{2y} \\ l_{3x}, l_{3y} \\ l_{4x}, l_{4y} \\ \vdots \\ l_{64}, l_{64y} \end{bmatrix}$$

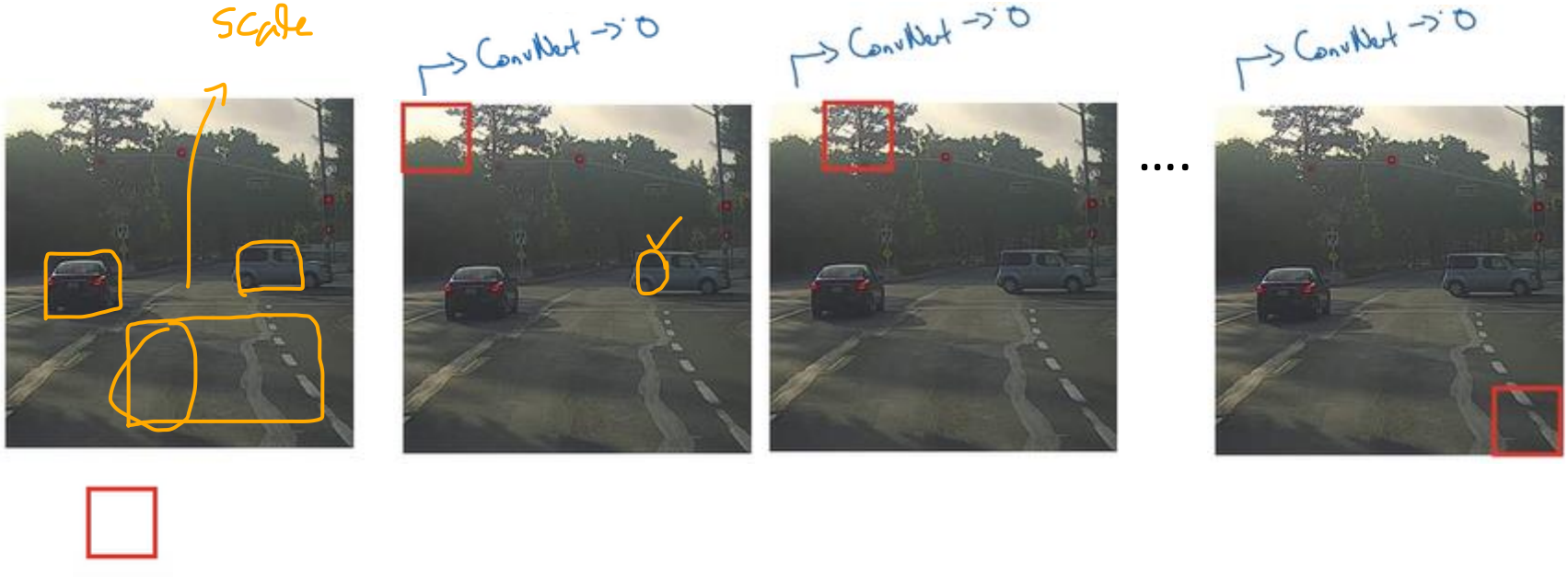
Object Detection

- Can we use Convolution Neural Net for Object Detection?
- Object Detection algorithm: Conv. Neural Net with sliding windows
- Car detection example



Sliding windows Detection (for Object detection)

- The trained ConvNet can be used in sliding windows detection. (Slide window across the entire image)



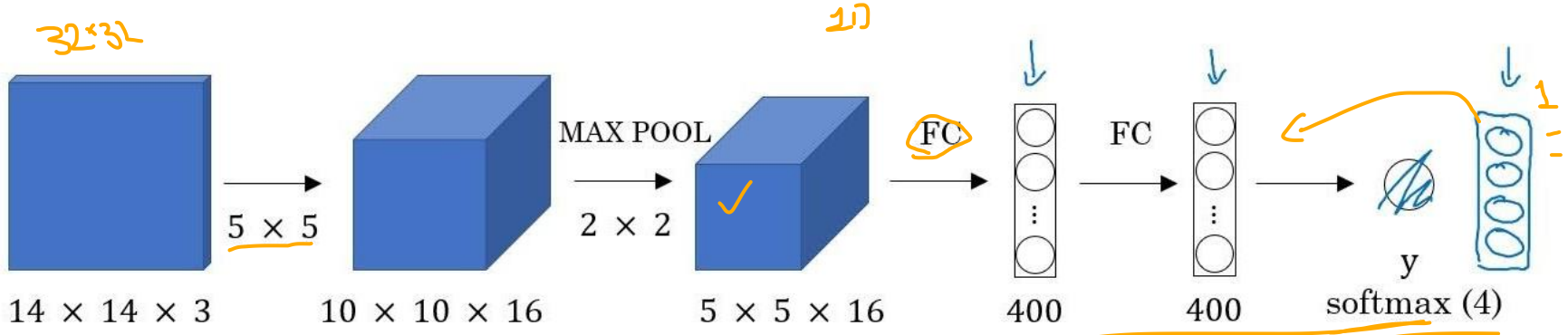
Sliding windows Detection (for Object detection)

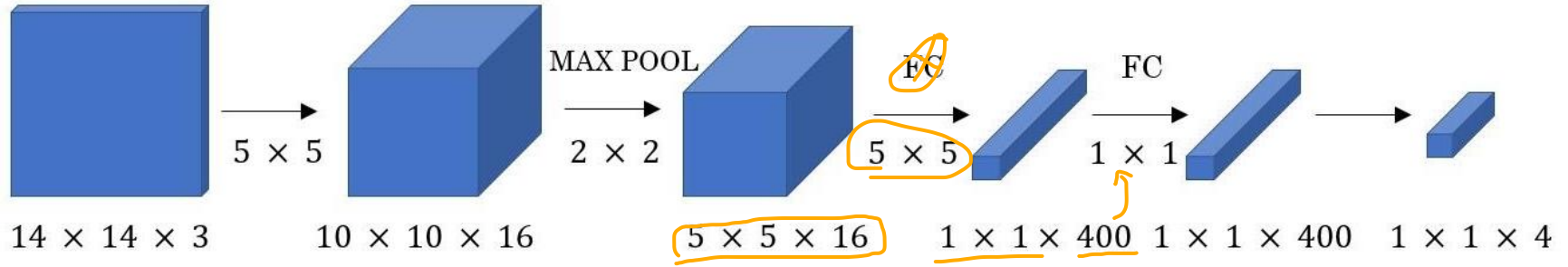
- Sliding window with Larger size

Convolution Implementation of Sliding Windows

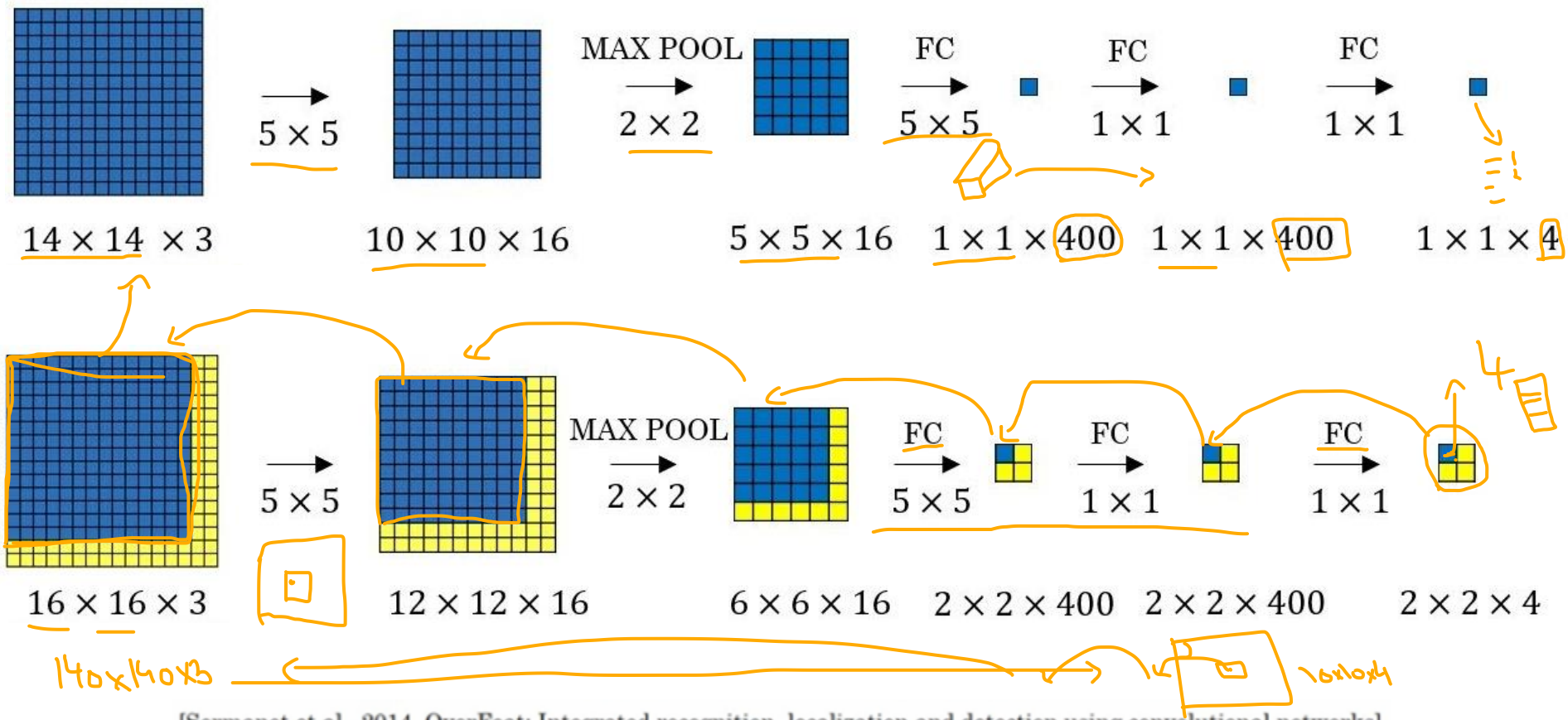
Convolution Implementation of Sliding Windows

- Fully connected layers are concerted in convolution layers



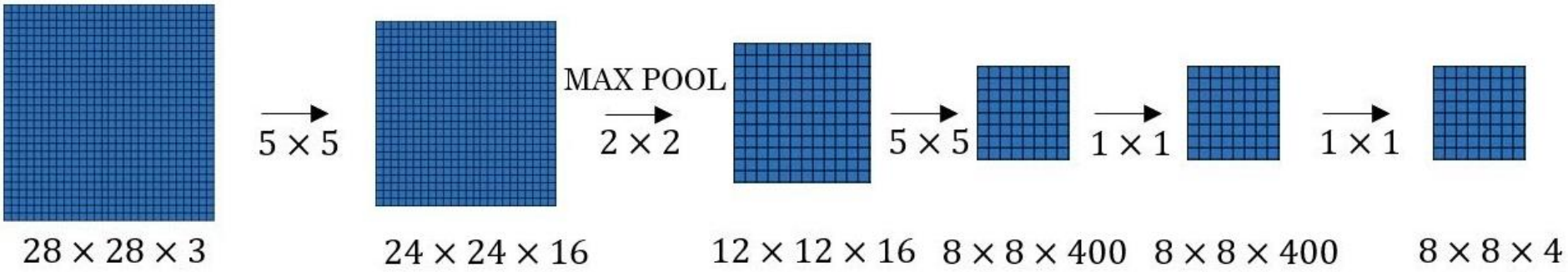


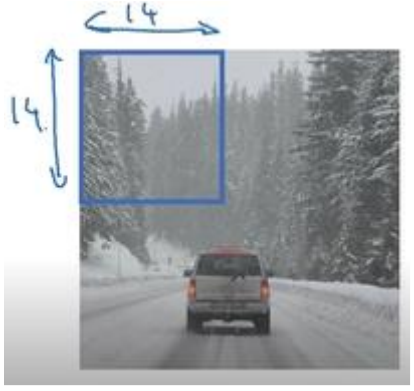
Convolution Implementation of Sliding Windows

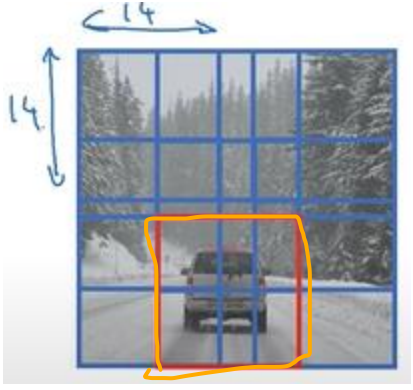


[Sermanet et al., 2014, OverFeat: Integrated recognition, localization and detection using convolutional networks]

Convolution Implementation of Sliding Windows



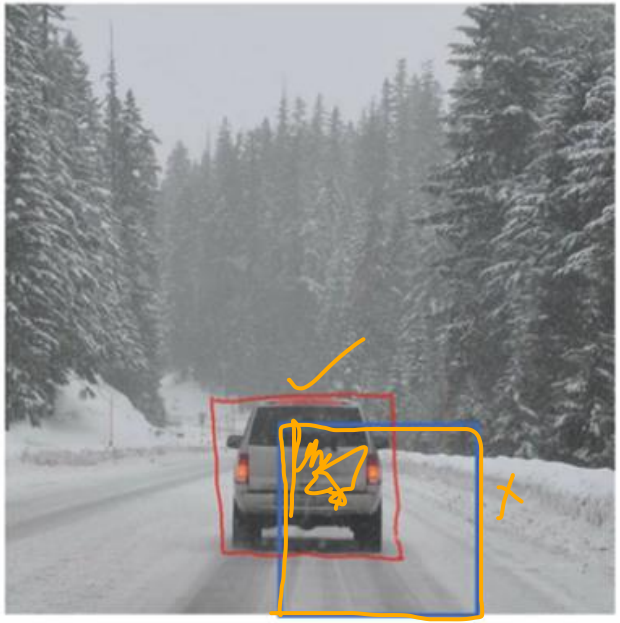




[Sermanet et al., 2014, OverFeat: Integrated recognition, localization and detection using convolutional networks]

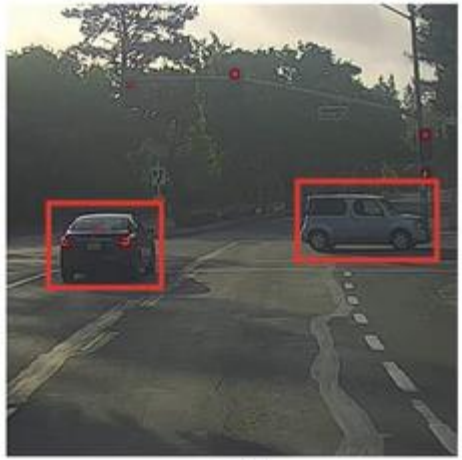
Convolution Implementation of Sliding Windows

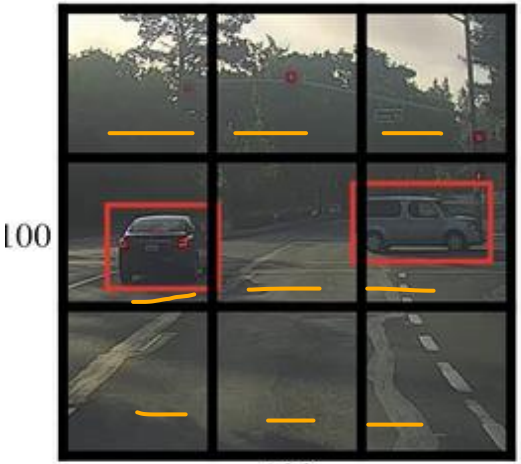
- Weakness:
  - Bounding boxes may not be really accurate



Accurate Object Detection (Bounding box prediction) using YOLO Algorithm

YOLO Algorithm





Labels for training

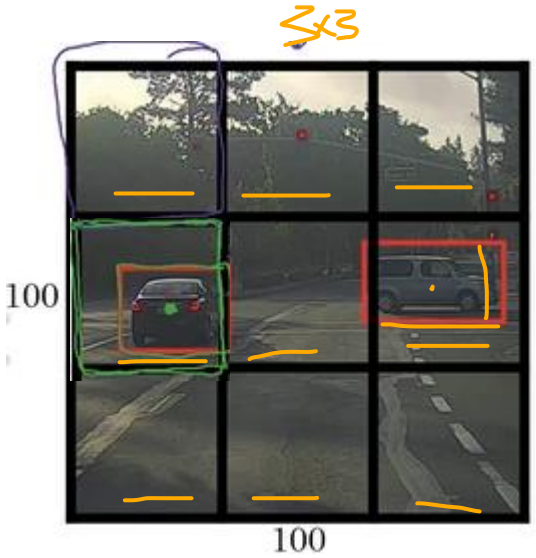
For each grid cell:

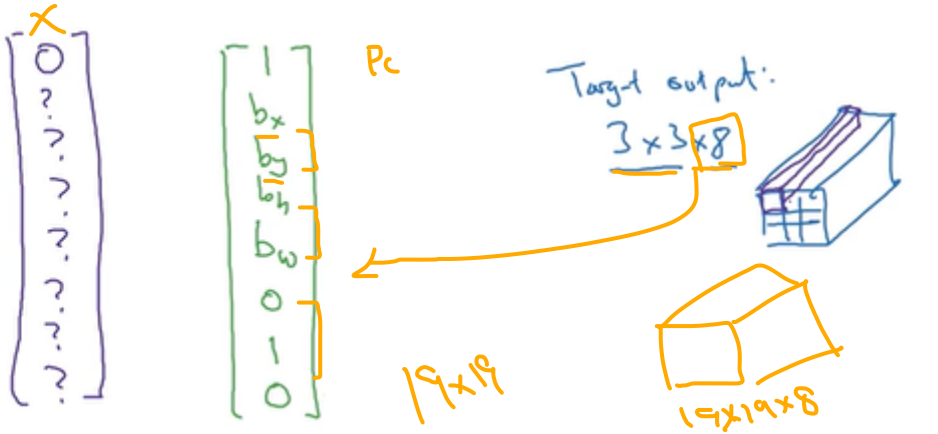
$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$

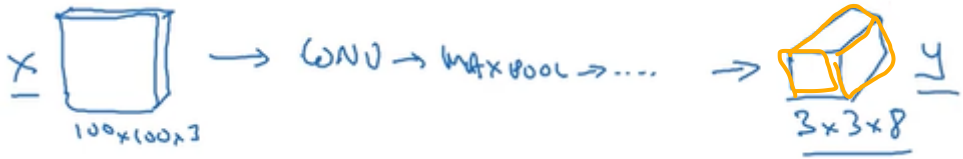
3x3 → 3x3x8

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

YOLO Algorithm

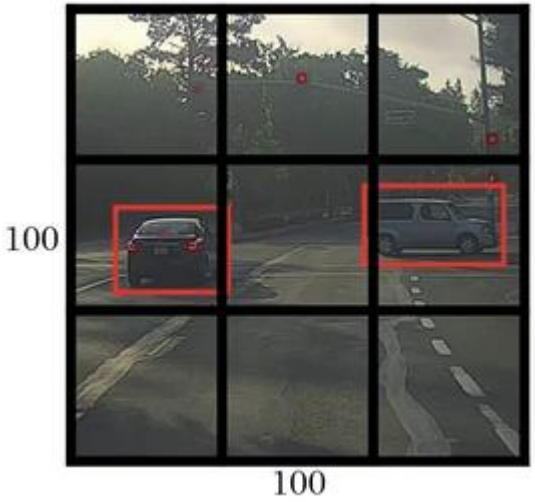


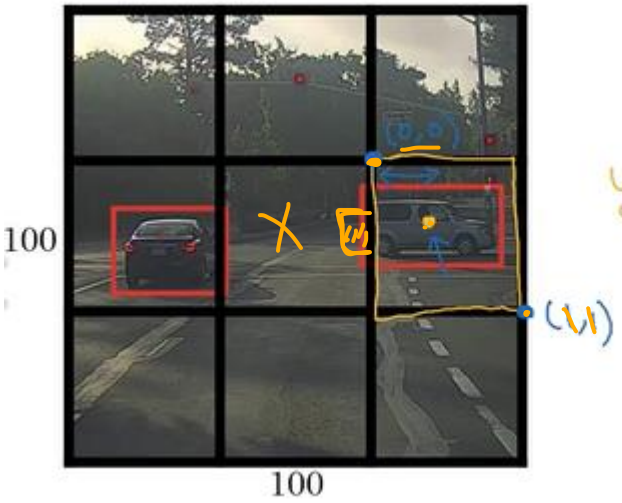




[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

How to specify the Bounding box?





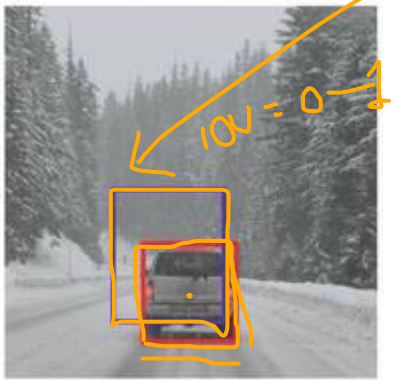
$$y = \begin{bmatrix} 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \\ 0.2 \end{bmatrix}$$

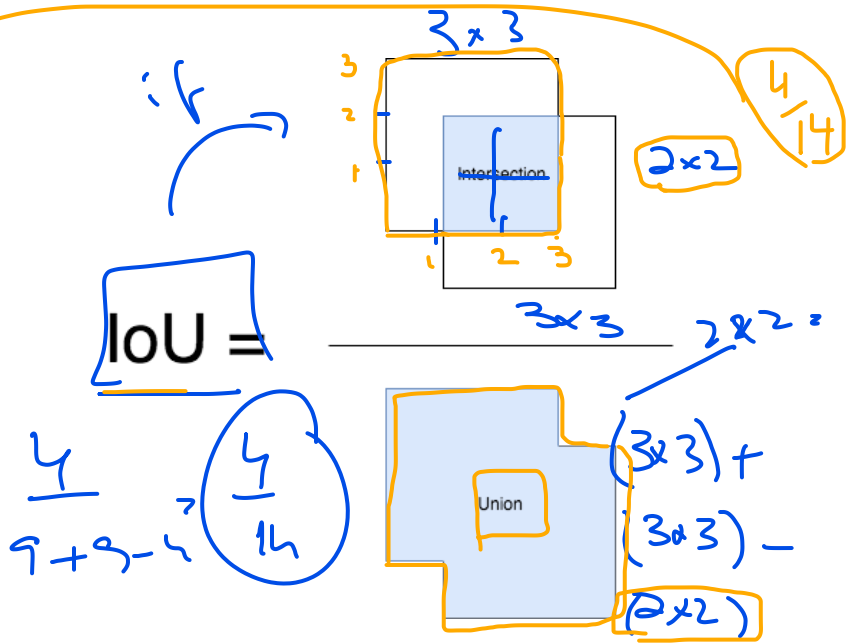
$\begin{bmatrix} 0.4 \\ 0.3 \end{bmatrix}$ between 0 and 1

$\begin{bmatrix} 0.9 \\ 0.5 \end{bmatrix}$ could be > 1

Evaluating Object Localization

Intersection Over Union (IoU): **It is a ratio between the intersection and the union of the predicted boxes and the ground truth boxes.** This stat is also known as the Jaccard Index and was first published by Paul Jaccard in the early 1900s.





Often used: correct if IoU >= 0.5

More generally, IoU is a measure of the overlap between two bounding boxes.

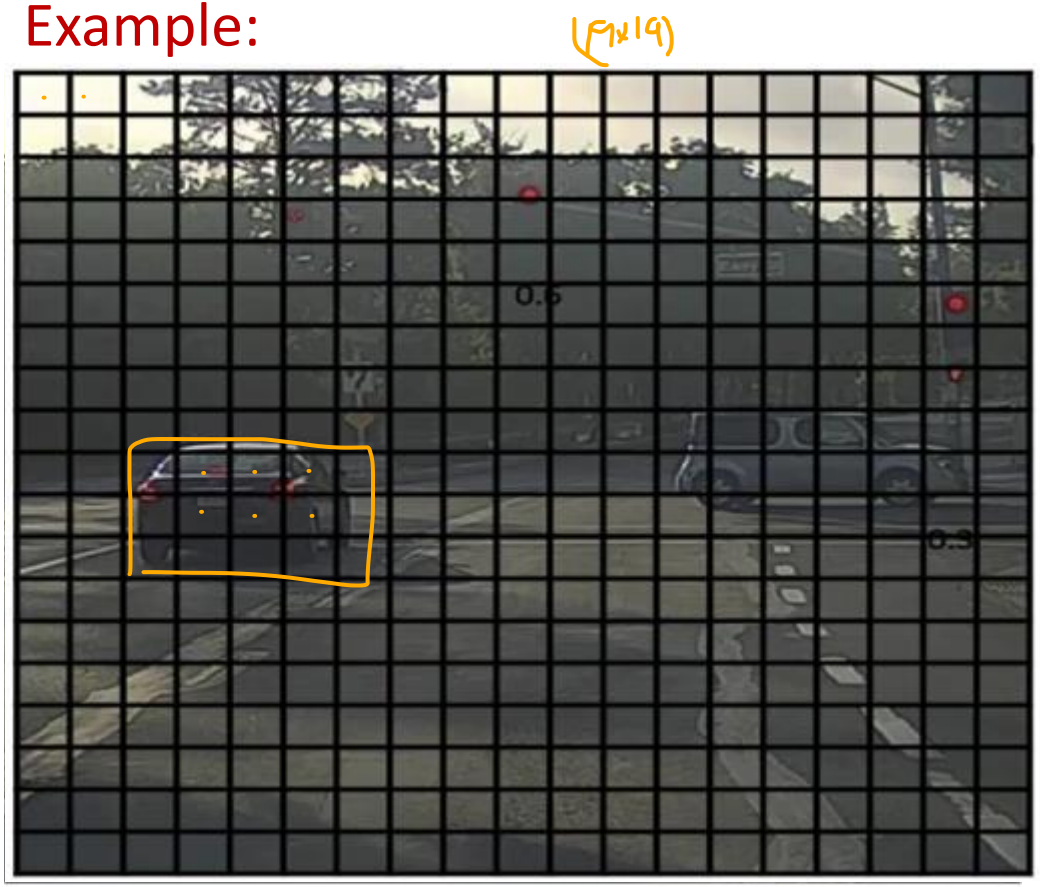
Object Detection:

Non-max Suppression

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

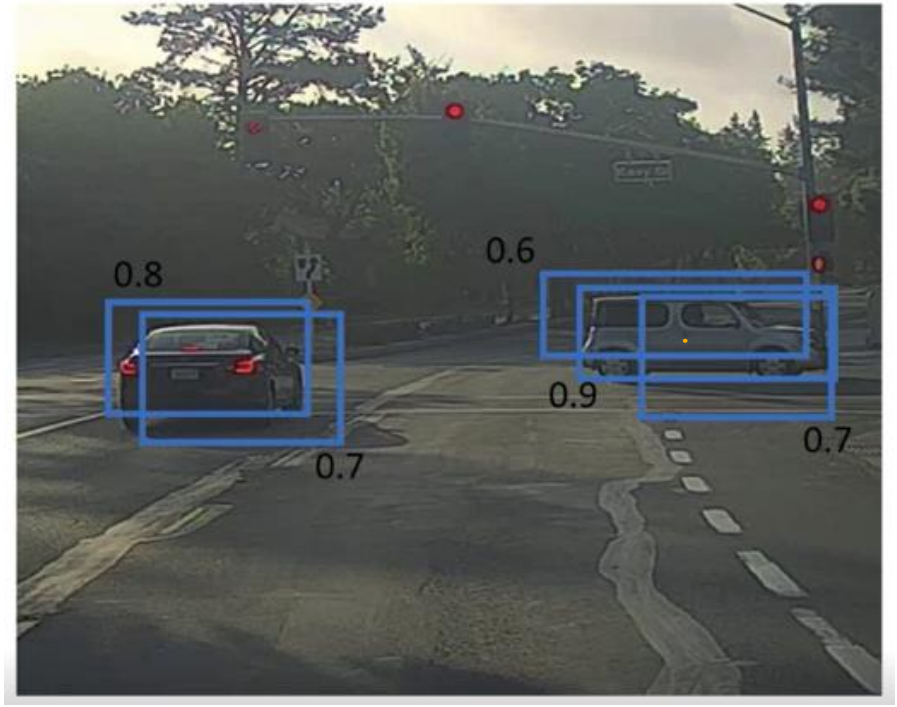
Non-max suppression

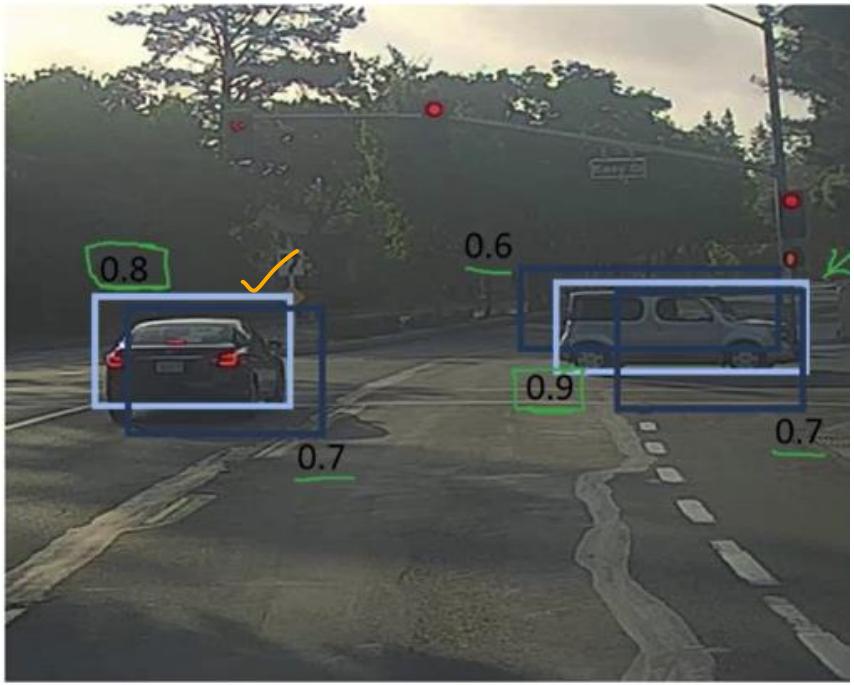
Example:



Non-max suppression

Example:

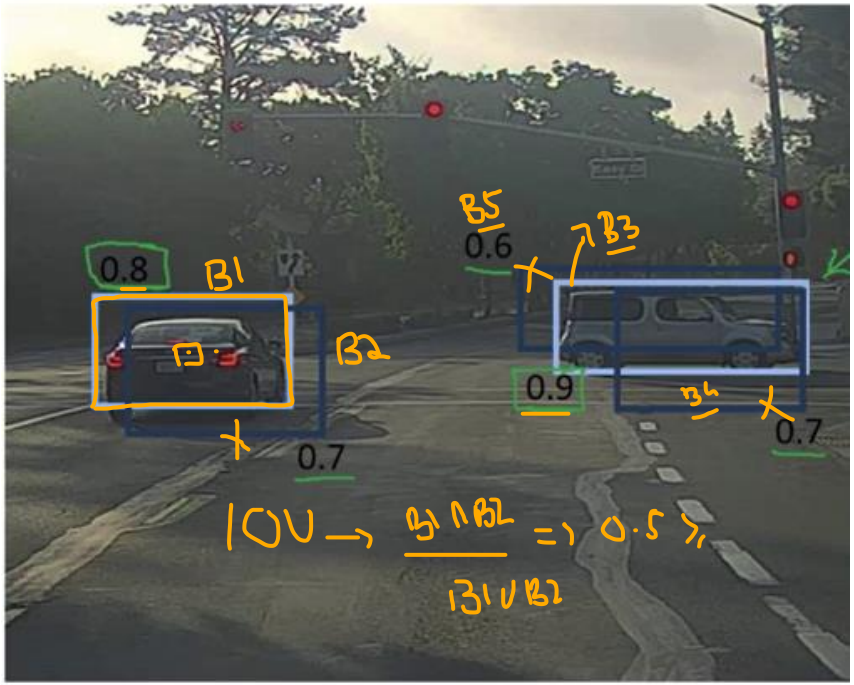


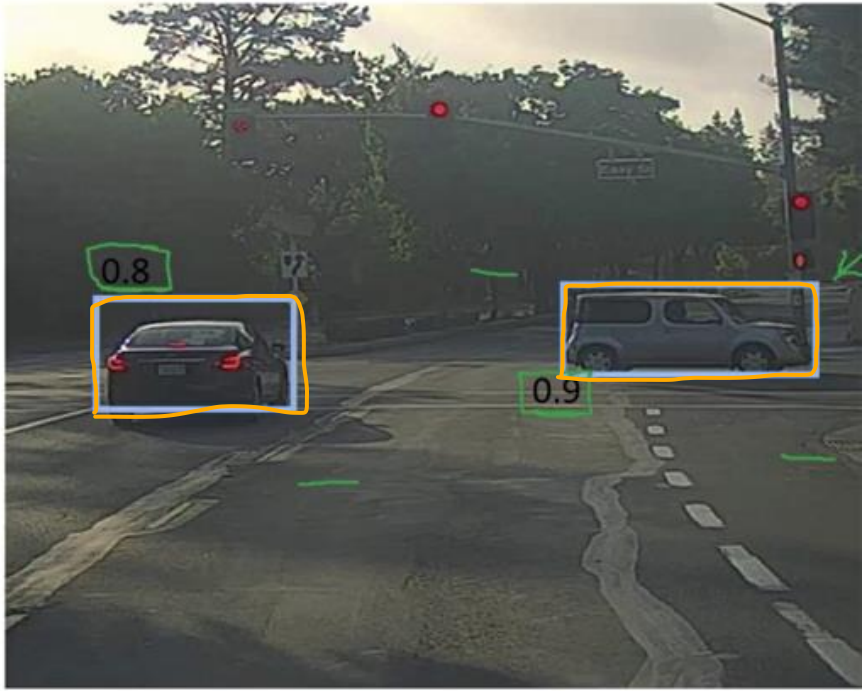


[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Non-max suppression

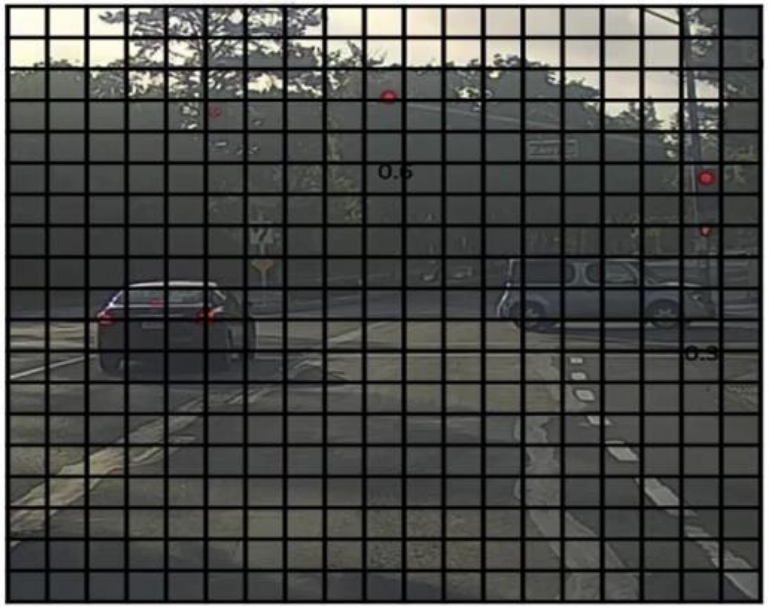
Example:





[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Non-max suppression algorithm



Each output prediction is: $\begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \end{bmatrix}$

Discard all boxes with $p_c \leq 0.6$

While there are any remaining boxes:

- Pick the box with the largest p_c
  Output that as a prediction.
- Discard any remaining box with IoU ≥ 0.5 with the box output in the previous step

Object Detection:

Anchor Boxes

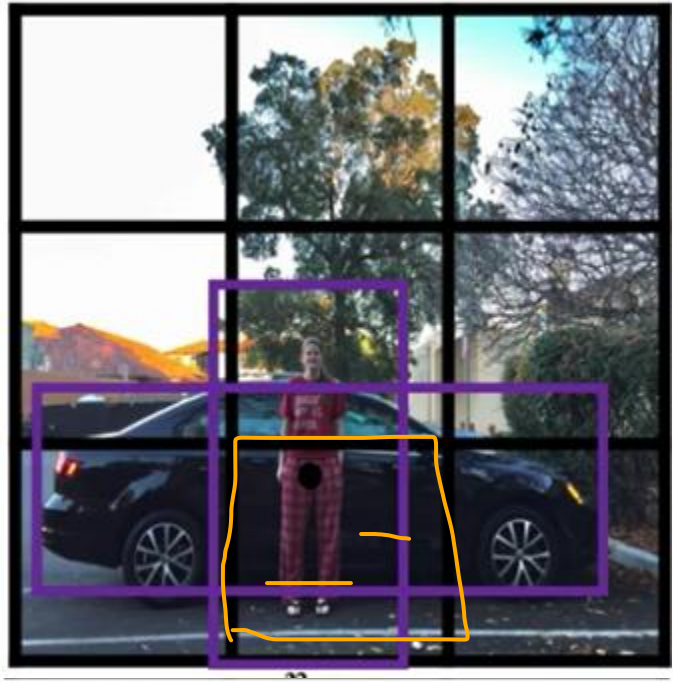
[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Anchor Boxes

Split image into boxes

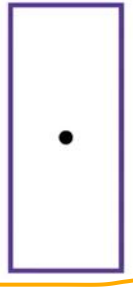
Overlapping Objects

3x3x8x2



$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

Anchor box 1:



Anchor box 2:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ \vdots \\ c_3 \end{bmatrix}$$

Anchor box 1

Anchor box 2

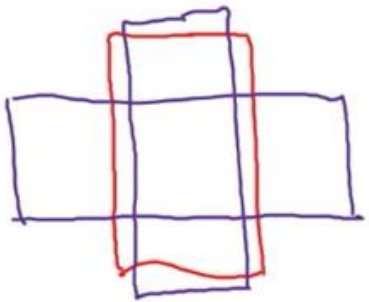
Multiple object centers in a single cell

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Anchor Box Algorithm

Previously:

Each object in training image is assigned to grid cell that contains that object's midpoint.



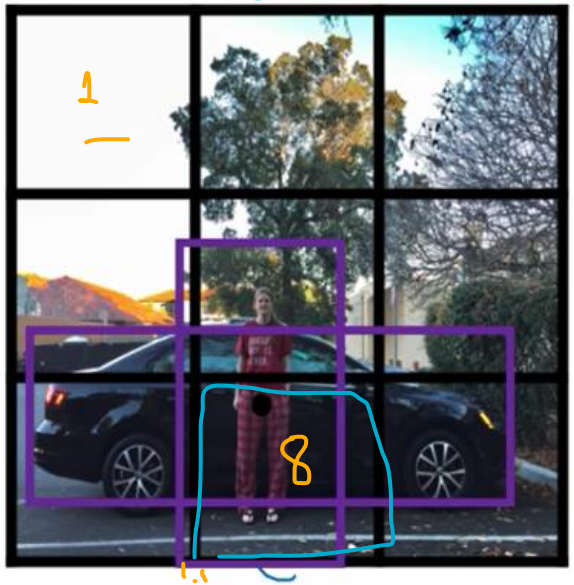
With two anchor boxes:

Each object in training image is assigned to grid cell that contains object's midpoint and anchor box for the grid cell with highest IoU.

(Grid cell, anchor box)

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Anchor Box Example



Anchor box 1: Anchor box 2:

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix}$$

3x3x16

A1

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Overall:

YOLO Object Detection Algorithm

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

YOLO Algorithm

Training

1. Pedestrian
2. Car
3. Motorcycle

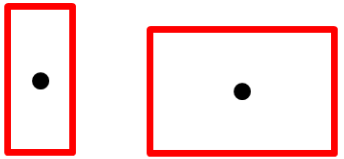
$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \quad y = \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \end{bmatrix} \quad y = \begin{bmatrix} 0 \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ ? \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$y = 3 \times 3 \times 16$

$y = 3 \times 3 \times \boxed{2} \times \boxed{8}$

anchors # parameters

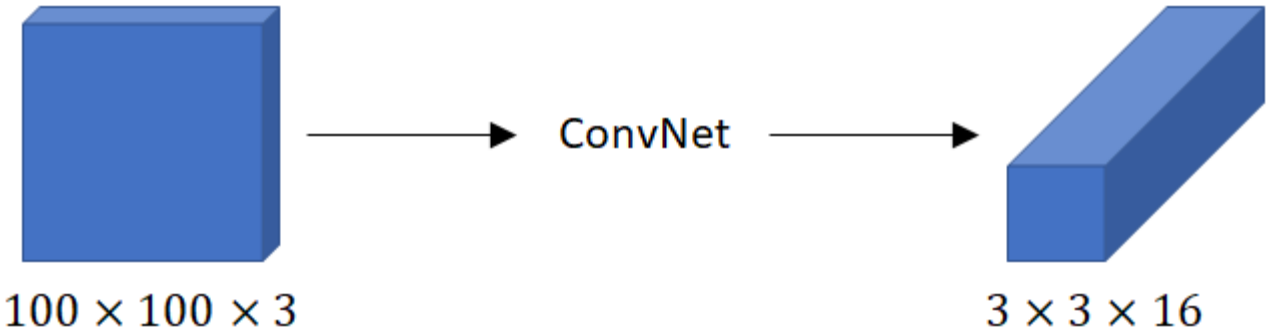
Anchor box 1: Anchor box 2:



[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

YOLO Algorithm

Training



[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

YOLO Algorithm

Prediction

$3 \times 3 \times 2 \times 8$

$$y = \begin{bmatrix} p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \\ p_c \\ b_x \\ b_y \\ b_h \\ b_w \\ c_1 \\ c_2 \\ c_3 \end{bmatrix} \quad \begin{bmatrix} 0 \\ \dots \\ \dots \\ \dots \\ \dots \\ \dots \\ \dots \\ \dots \\ 0 \\ \dots \\ \dots \\ \dots \\ \dots \\ \dots \\ \dots \\ \dots \end{bmatrix} \quad \begin{bmatrix} 0 \\ \dots \\ \dots \\ \dots \\ \dots \\ \dots \\ \dots \\ \dots \\ 1 \\ b_x \\ b_y \\ b_h \\ b_w \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

YOLO Algorithm

Prediction: How do we run a non-max suppression?

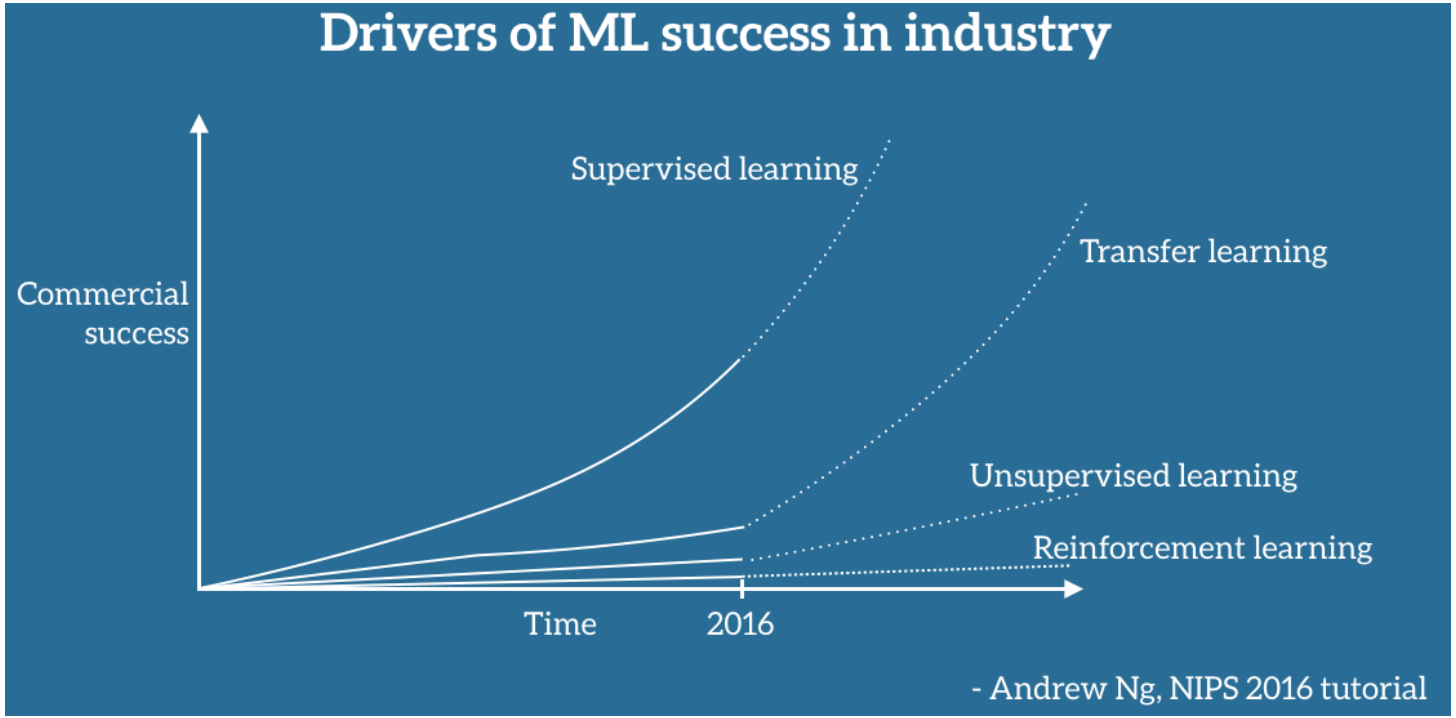
- For each cell get 2 predicted bounding boxes
- Get rid of low probability predictions
- For each class (pedestrian, car, motorbike), use non-max suppression to generate final predictions.

[Redmon et al., 2015, You Only Look Once: Unified real-time object detection]

Transfer Learning

- Humans have an inherent ability to transfer knowledge across tasks.
- What we acquire as knowledge while learning about one task, we utilize in the same way to solve related tasks.
- The more related the tasks, the easier it is for us to transfer, or cross-utilize our knowledge. Some simple examples would be:
  - now how to ride a motorbike ➦ Learn how to ride a car
  - Know how to play classic piano ➦ Learn how to play jazz piano
  - Know math and statistics ➦ Learn machine learning

Motivation of Transfer Learning



Drivers of ML industrial success according to Andrew Ng

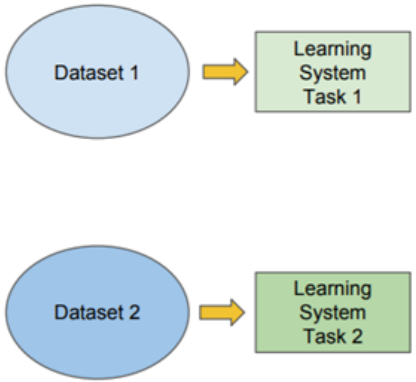
Transfer Learning

- In their famous book, *Deep Learning*, Goodfellow et al. refer to transfer learning in the context of generalization. Their definition is as follows: *"Situation where what has been learned in one setting is exploited to improve generalization in another setting. "*
- **Transfer learning (TL)** is a research problem in machine learning (ML) that focuses on storing knowledge gained while solving one problem and applying it to a different but related problem. For example, knowledge gained while learning to recognize cars could apply when trying to recognize trucks.

Traditioanal ML vs Transfer Learning

Traditional ML

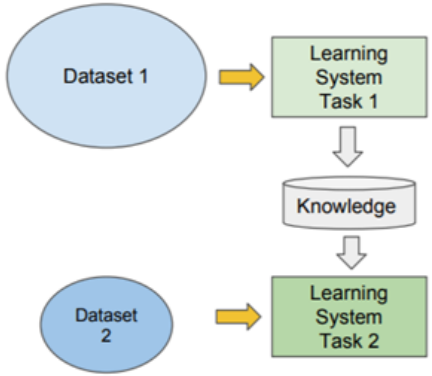
- Isolated, single task learning:
  - Knowledge is not retained or accumulated. Learning is performed w.o. considering past learned knowledge in other tasks



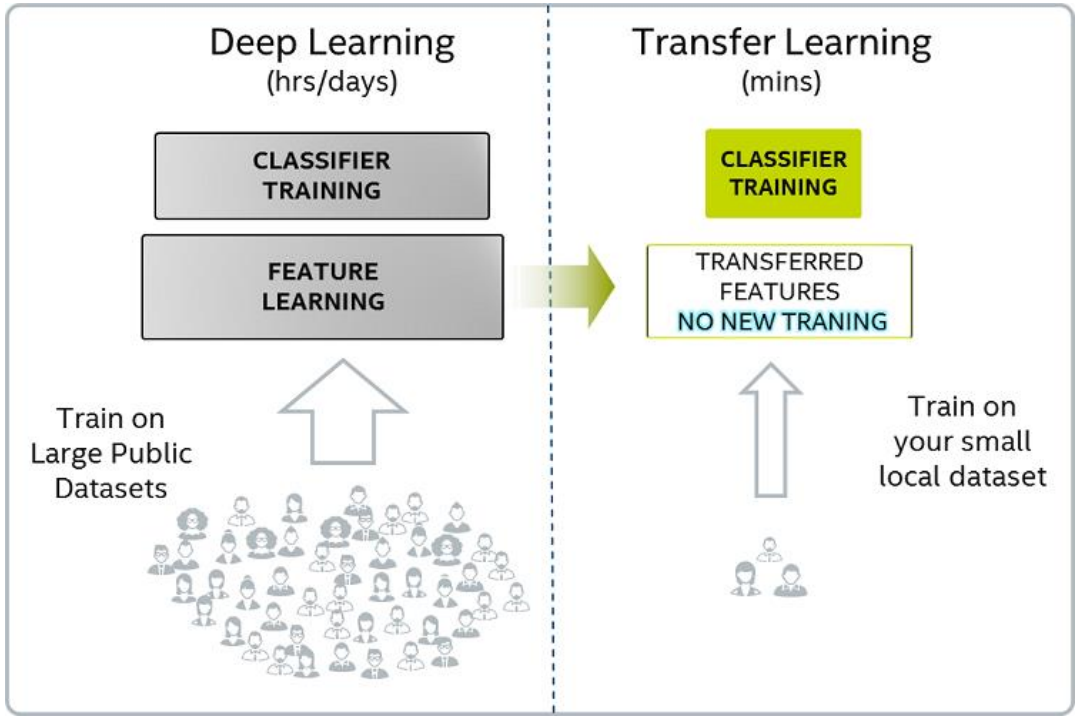
vs

Transfer Learning

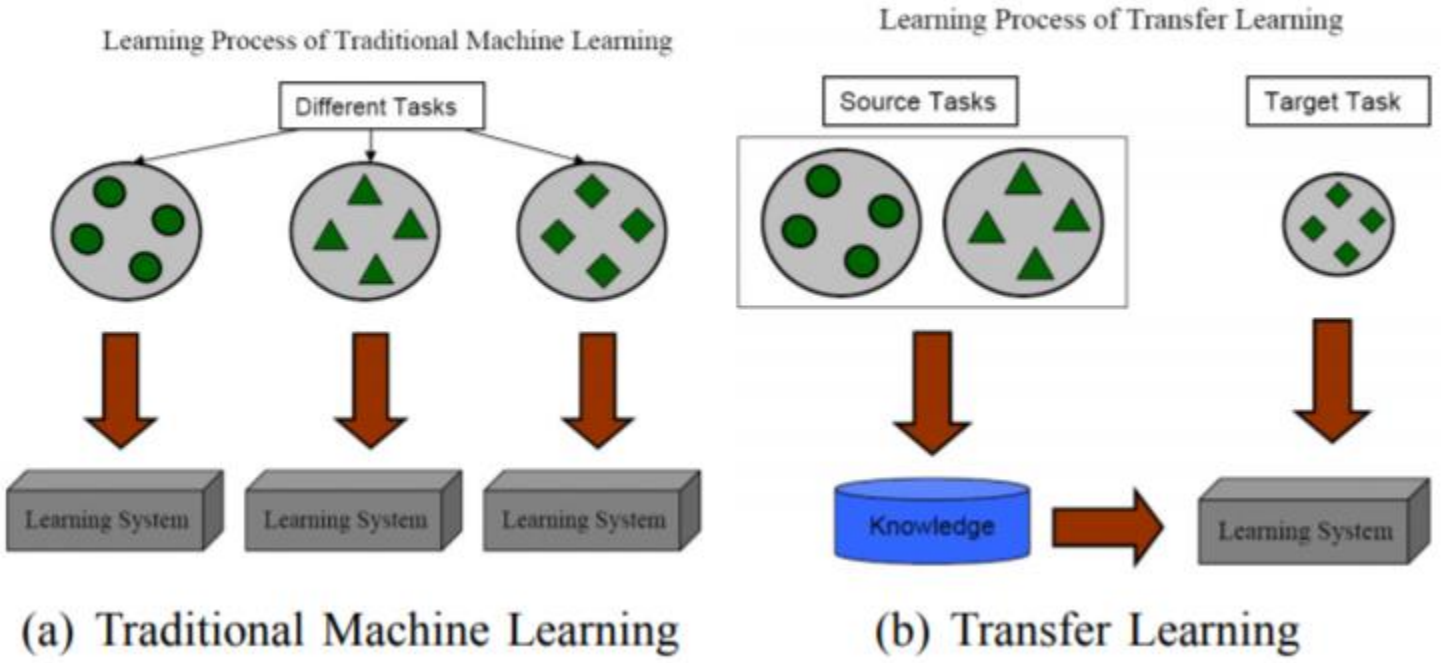
- Learning of a new tasks relies on the previous learned tasks:
  - Learning process can be faster, more accurate and/or need less training data



Why Transfer Learning?

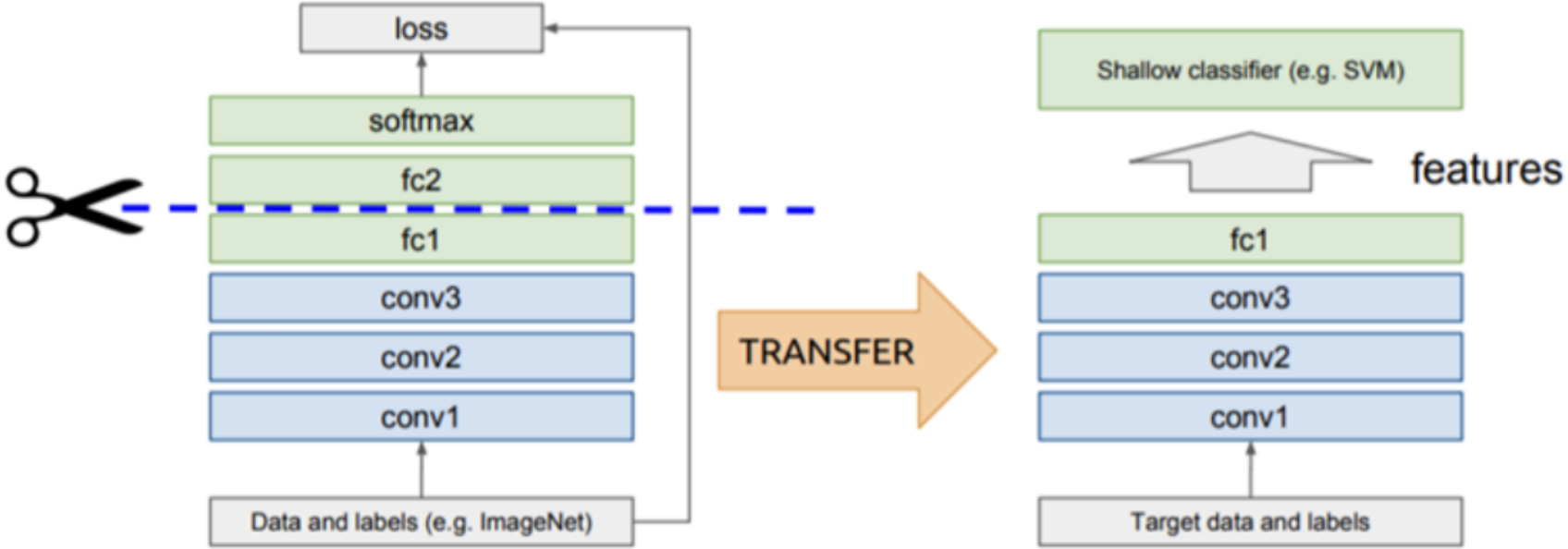


Transfer Learning



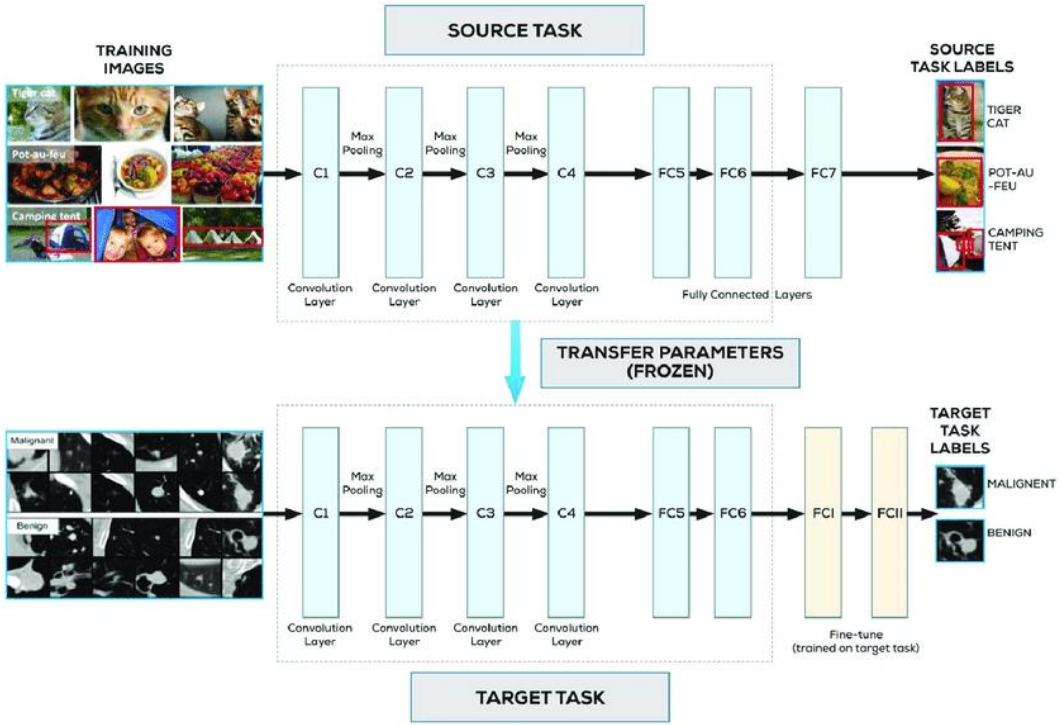
Fully Trained Network

Transfer Learning

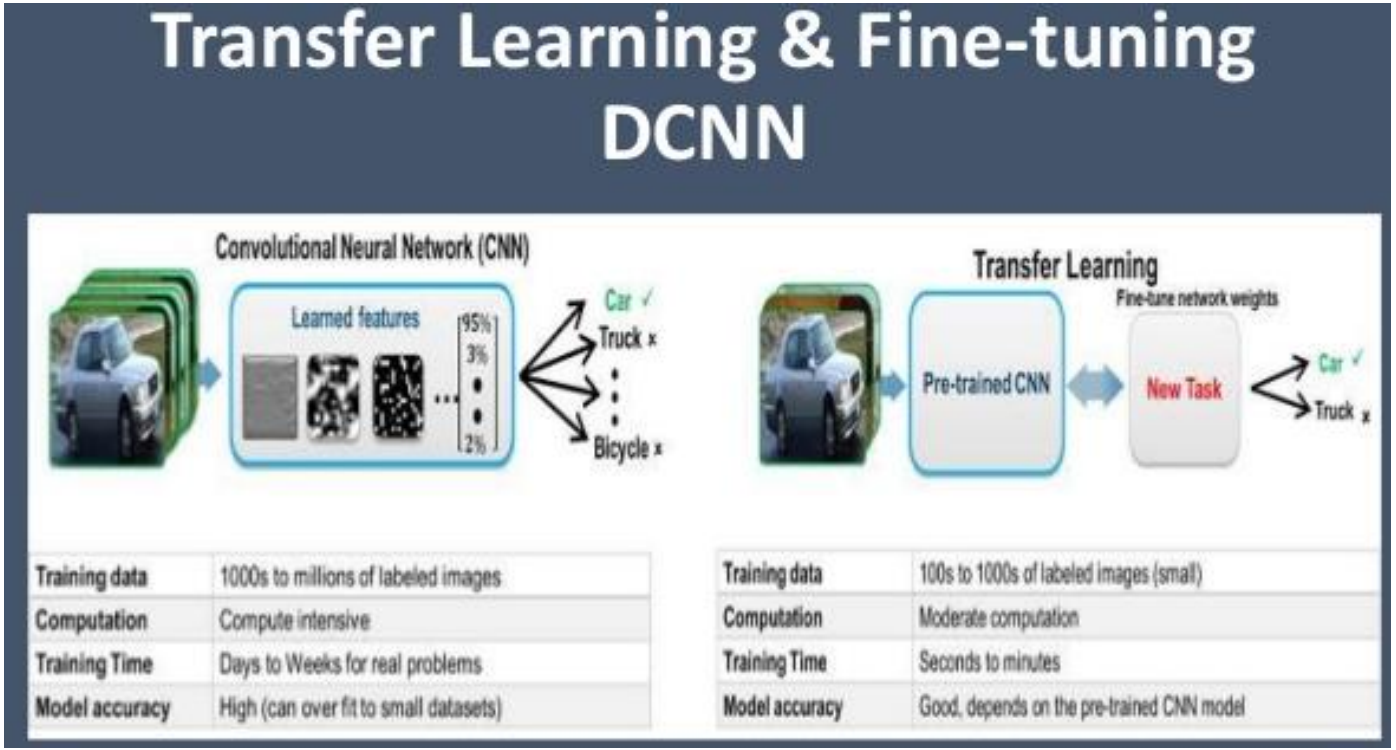


Transfer Learning

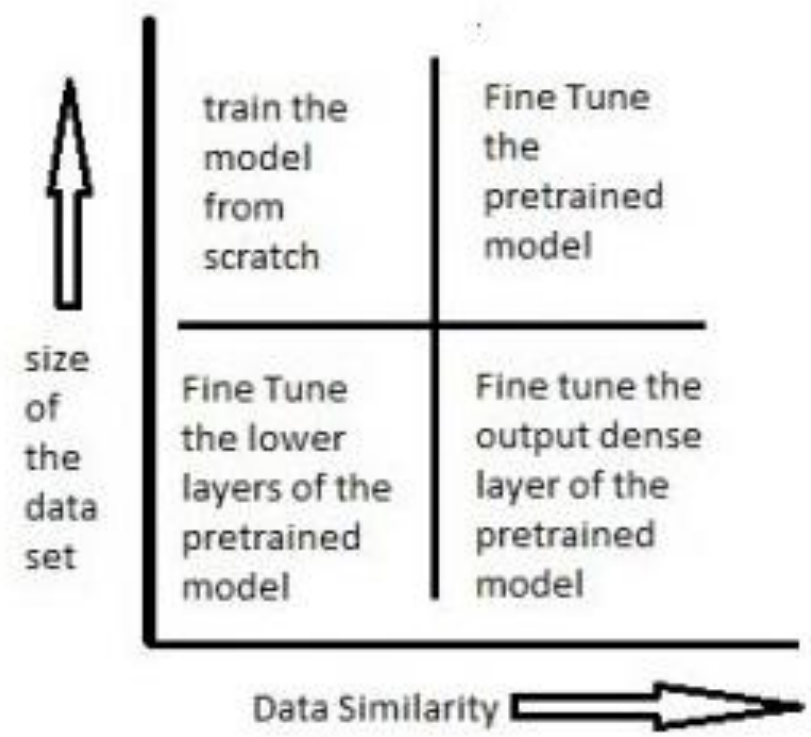
- There are different options, which layers should be frozen and which should be retrained or fine tuned.



Transfer Learning



General strategy



Transfer Learning: Controlling layers which need to be frozen